FATECalçados — Para que serve cada pasta

Organização do código · Desafio de Inovação Data System 2026

Cada pasta do projeto tem uma responsabilidade só. Assim, quem precisa mudar alguma coisa sabe onde procurar, e uma mudança numa parte não quebra as outras. Este documento explica o papel de cada pasta; o *Guia do Projeto* (guia-do-projeto.pdf) lista as funções e os endpoints.

1. Raiz do repositório

`backend/`

Responsabilidade: o servidor: guarda os dados no PostgreSQL, aplica as regras do programa de fidelidade e responde a API em /api/. É um projeto Django.

`frontend/`

Responsabilidade: tudo que o usuário vê no navegador: caixa (PDV), painel da gerência e área do cliente. É um app React + Vite em TypeScript.

`docs/`

Responsabilidade: os documentos do projeto em PDF (regras de pontos e bônus, onde usamos IA, guia do projeto e este arquivo).

`docker-compose.yml`

Responsabilidade: subir os três serviços juntos (banco, backend e frontend) com um só comando.

Regra: nunca usar `docker compose down -v`, que apaga o banco.

2. Backend

`backend/config/`

Responsabilidade: a configuração geral do Django: banco de dados, apps instalados, segurança (CSRF, origens confiáveis) e as URLs raiz, que juntam o /admin/ com as rotas da aplicação.

Exemplos: settings.py, urls.py, wsgi.py, asgi.py

`backend/src/`

Responsabilidade: o sistema em si (uma única app Django). Dentro dela os arquivos são separados por camada, e cada camada só chama a de baixo:

Exemplos: regras.py (números do programa, sem banco) -> **services.py** (única parte que grava regras no banco) -> **consultas.py / indicadores.py / acoes.py** (só leitura, para listas e dashboard) -> **views.py** (recebe a requisição e devolve JSON). **models.py** define as tabelas e **admin.py** o painel /admin.
Regra: mudança de regra começa em regras.py; views não têm regra de negócio.

`backend/src/migrations/`

Responsabilidade: o histórico das mudanças nas tabelas do banco. O Django usa esses arquivos para criar ou atualizar as tabelas com `manage.py migrate`.

Exemplos: 0001_initial.py, 0002_pdv.py, 0003_acesso_do_cliente.py
Regra: são geradas com makemigrations; não se edita uma migração já aplicada.

`backend/src/management/commands/`

Responsabilidade: comandos próprios do sistema que rodam pelo terminal com manage.py.

Exemplos: popular_dados (cria dados de exemplo passando pelas mesmas regras do caixa) e **expirar_pontos** (vence os lotes de pontos com mais de 12 meses)

`backend/src/tests/`

Responsabilidade: os testes automáticos do backend. Garantem que as regras continuam certas depois de qualquer mudança.

Exemplos: test_regras.py (níveis, bônus, resgate), test_pdv.py (vendas do caixa), test_conta.py (área do cliente), test_telas.py (números do dashboard)
Regra: toda regra de regras.py/services.py tem um teste correspondente.

3. Frontend

`frontend/public/`

Responsabilidade: arquivos servidos do jeito que estão, sem passar pelo build, como o ícone da aba e a logo.

Exemplos: favicon.svg, fatec-icon.svg

`frontend/src/`

Responsabilidade: o código do app. Os arquivos da raiz iniciam o app (main.tsx, App.tsx), definem os estilos globais e o tema (index.css), os tipos compartilhados (types.ts), as regras de pontos do caixa (regras.ts) e qual caixa este PDV é (config.ts).

`src/routes/`

Responsabilidade: definir quais endereços existem e qual tela abre em cada um (/pdv, /dashboard, /loja…), e qual layout envolve cada grupo de telas.

Exemplos: index.tsx

`src/layouts/`

Responsabilidade: a moldura que fica em volta das telas: barra lateral, cabeçalho e o espaço onde a tela aparece. Também monta os providers que guardam estado entre as telas.

Exemplos: MainLayout (telas da loja, com tour guiado e busca) e **ClienteLayout** (área do cliente; sem login manda para /login)

`src/pages/`

Responsabilidade: as telas. Cada subpasta é uma área do sistema e guarda a tela e as peças que só ela usa.

Exemplos: inicio (landing, login, cadastro), **pdv** (caixa), **dashboard**, **clientes**, **vendas**, **acoes** e **conta** (loja online e Minha conta)
Regra: as páginas só compõem: a lógica fica em hooks/ e as chamadas à API em services/.

`src/components/`

Responsabilidade: componentes reaproveitados por várias telas, separados por tipo:

Exemplos: ui/ peças visuais básicas (Button, Card, Modal, Pill, LevelBadge, Pagination, Toast); **layout/** barra lateral, lista de menus, busca Ctrl/Cmd K, tour guiado e painel de acessibilidade; **charts/** gráficos do dashboard
Regra: um componente usado por uma tela só fica na pasta da própria tela, em pages/.

`src/hooks/`

Responsabilidade: a lógica das telas, separada do visual. Um hook guarda estado e comportamento que a tela só usa, sem saber como funciona por dentro.

Exemplos: useCarrinho (itens e totais do carrinho), useFilaVendas (vendas offline), useCarregar (carregar da API com carregando/erro/tentar de novo), useAtalhosPdv (F2/F10), useTema (claro/escuro), useTour (tour guiado)

`src/contexts/`

Responsabilidade: o estado que precisa sobreviver quando o usuário troca de tela, porque as páginas são desmontadas ao sair delas. Cada contexto tem um arquivo .ts (contexto + hook) e um *Provider.tsx.

Exemplos: Caixa (online/offline, fila e venda em andamento), **Acoes** (ações marcadas como feitas), **Conta** (sessão e carrinho do cliente), **Toast** (avisos rápidos)

`src/services/`

Responsabilidade: falar com o mundo de fora da tela. **api/** é o único lugar que chama o backend: um módulo por área (produtos, clientes, vendas, acoes, dashboard, conta) que também traduz os nomes dos campos da API (português) para os usados nos componentes (inglês). **filaVendas.ts** guarda as vendas offline no navegador.

Exemplos: fetchProducts, sendSale, fetchDashboard, signIn
Regra: nenhuma tela usa fetch direto; endpoint novo entra no módulo da área e é exportado em api/index.ts.

`src/utils/`

Responsabilidade: funções puras (sem estado e sem tela) usadas em dois ou mais lugares.

Exemplos: format.ts (dinheiro, números, datas, plural) e cpf.ts (máscara e validação de CPF)
Regra: função usada num lugar só fica ao lado de quem usa; vai para utils quando um segundo lugar precisar.

`src/testes/`

Responsabilidade: os testes da camada de API do frontend. Rodam cada função de services/api contra um fetch falso, com respostas simuladas em **dados/**.

Exemplos: api.test.ts, apiFalsa.ts, dados/clientes.ts
Regra: os testes de lógica pura (regras.ts, utils, csv) ficam ao lado do arquivo testado; não há testes de componentes.

`src/assets/`

Responsabilidade: imagens e arquivos importados pelo código e empacotados no build (diferente de public/).

Exemplos: loja/hero.jpg